Blackwell Nvfp4 Kernel Hackathon Journey

Dec 2, 2025

My Blackwell NVFP4 Kernel Hackathon Journey

I recently joined the Blackwell NVFP4 Kernel Hackathon hosted by GPU Mode. This blog shares my kernel optimization journey for Problem 1: NVFP4 Batched GEMV, where I achieved a final submission of **22.392µs** on the leaderboard.

Problem Description

The challenge was to implement a batched matrix-vector multiplication kernel optimized for NVIDIA B200. Given a tuple of tensors `(a, b, sfa, sfb, c)`:

- `a` is M × K × L in K-major order in nvfp4(e2m1)
- `b` is 1 × K × L in K-major order in nvfp4(e2m1)
- `sfa` is M × (K // 16) × L in K-major order in fp8(e4m3fnuz)
- `sfb` is 1 × (K // 16) × L in K-major order in fp8(e4m3fnuz)
- `c` is M × 1 × L in fp16

Matrix sizes: `M` is divisible by `mma_tiler_mn[0]`, `K` is divisible by 64. The ranking criteria is the geometric mean of benchmark results.

My Journey Overview

I explored two parallel paths:

1. **CuTe DSL Path**: Started from the provided template (100µs) → optimized to ~33µs. Still had room to optimize, but switched gears to CUDA which felt more promising to me considering my CuTe DSL skill gap and gave me more control over what happens under the hood.
2. **CUDA Path**: Started from scratch (2000µs) → optimized to ~22.3µs

This was my first time using CuTe DSL — I typically write kernels in CUDA at work, so my CuTe DSL skill is limited and I just started learning during the hackathon.

Part 1: CuTe DSL Journey (100µs → ~33µs)

Starting Point: Template Implementation (100µs)

The organizers provided a template implementation using CuTe DSL that achieved ~100µs. The core computation loop:

```
for k_tile in range(k_tile_cnt):
    tAgA = gA_mkl[tidx, None, bidx, k_tile, bidz]
    tBgB = gB_nkl[0, None, bidy, k_tile, bidz]
    tAgSFA = gSFA_mkl[tidx, None, bidx, k_tile, bidz]
    tBgSFB = gSFB_nkl[0, None, bidy, k_tile, bidz]

    # Load and convert to float32
    a_val = a_val_nvfp4.to(cutlass.Float32)
    b_val = b_val_nvfp4.to(cutlass.Float32)
    sfa_val = sfa_val_fp8.to(cutlass.Float32)
    sfb_val = sfb_val_fp8.to(cutlass.Float32)

    # Compute
    for i in cutlass.range_constexpr(mma_tiler_mnk[2]):
        res += tArA[i] * tArSFA[i] * tBrB[i] * tBrSFB[i]
```

Optimization 1: Avoid Loading Duplicate Scales

The original code loaded scale factors redundantly. Since 16 FP4 elements share one scale factor, we can load scales more efficiently:

Before:

```
tAgSFA = gSFA_mkl[tidx, None, bidx, k_tile, bidz]
tBgSFB = gSFB_nkl[0, None, bidy, k_tile, bidz]
```

After:

```
tAgSFA = gSFA_mkl[tidx, (0, None), bidx, k_tile, bidz]
tBgSFB = gSFB_nkl[0, (0, None), bidy, k_tile, bidz]
```

This change reduces unnecessary scale factor loads.

Optimization 2: Use Float16 for A/B Register Storage

Storing loaded A and B values as float16 instead of float32 saves register pressure:

Before:

```
tArA = cute.make_rmem_tensor_like(tAgA, cutlass.Float32)
tBrB = cute.make_rmem_tensor_like(tBgB, cutlass.Float32)
```

After:

```
tArA = cute.make_rmem_tensor_like(tAgA, cutlass.Float16)
tBrB = cute.make_rmem_tensor_like(tBgB, cutlass.Float16)
```

Optimization 3: Avoid Repeated Scale Product Calculation

Instead of computing `scale_a * scale_b` for every element, compute it once per scale factor block:

Before:

```
for i in cutlass.range_constexpr(mma_tiler_mnk[2]):
    res += tArA[i] * tArSFA[i] * tBrB[i] * tBrSFB[i]
```

After:

```
for sf_block in cutlass.range_constexpr(num_sf_blocks):
    scale_prod = tArSFA[sf_block] * tBrSFB[sf_block]
    base = sf_block * sf_vec_size
    for offset in cutlass.range_constexpr(sf_vec_size):
        element_idx = base + offset
        res += scale_prod * (tArA[element_idx] * tBrB[element_idx])
```

Optimization 4: Further Reduce Multiply Operations with Scale

Thanks to Chunan Zeng for pointing this out — we can accumulate the element products first, then multiply by the scale once:

```
for sf_block in cutlass.range_constexpr(mma_tiler_mnk[2] // sf_vec_size):
    tmp = cute.zeros_like(tCgC, cutlass.Float32)
    base = sf_block * sf_vec_size
    for offset in cutlass.range_constexpr(sf_vec_size):
        tmp += tArA[base + offset] * tBrB[base + offset]
    res += tArSFA[sf_block] * tBrSFB[sf_block] * tmp
```

Optimization 5: Thread Collaboration with Shared Memory

Thanks to Simon Veitner's blog on NVFP4 GEMV, by using multiple threads to collaborate on computing each output element, with shared memory for partial sum reduction:

```
threads_per_m = 32
threads_per_k = 512 // threads_per_m

# Allocate shared memory for partial sums
row_sum_buffer = allocator.allocate_tensor(
    element_type=cutlass.Float32,
    layout=cute.make_layout((threads_per_m, threads_per_k), stride=(threads_per_k
)

# Each thread processes different K tiles
for k_tile in range(tidy, k_tile_cnt, threads_per_k):
    # ... computation ...

# Store partial result to shared memory
row_sum_buffer[(tidx, tidy)] = res[0]
cute.arch.sync_threads()

# First thread per row reduces and writes output
if tidy == 0:
    out = cute.zeros_like(tCgC, cutlass.Float32)
    for i in cutlass.range_constexpr(threads_per_k):
        out += row_sum_buffer[(tidx, i)]
    tCgC.store(out.to(cutlass.Float16))
```

CuTe DSL Attempts That Didn't Help

I also tried:

- **Double buffering with async copy**: Made things slower
- **Loading entire B to shared memory**: No improvement

After reaching ~33µs with CuTe DSL, I felt uncertain about whether the data loading was truly vectorized and coalesced under the hood. Due to my limited experience with CuTe DSL, I decided to start fresh with CUDA for more control.

Part 2: CUDA Journey (2000µs → ~22.3µs)

Starting Point: Naive CUDA Implementation (2000µs)

My initial CUDA implementation manually decoded FP4 and FP8 values:

```
__device__ __forceinline__ float decode_fp4_e2m1(uint8_t packed, int lane)
{
    uint8_t nibble = (lane == 0) ? (packed & 0x0F) : (packed >> 4);
    float sign = (nibble & 0x8) ? -1.0f : 1.0f;
    unsigned int exp_raw = (nibble >> 1) & 0x3u;
    unsigned int mant_bit = nibble & 0x1u;
    float mant_f = (exp_raw == 0) ? (mant_bit * 0.5f) : (1.0f + mant_bit * 0.5f);
    int exp = (exp_raw == 0) ? 0 : (static_cast<int>(exp_raw) - 1);
    return sign * mant_f * ldexpf(1.0f, exp);
}

__device__ __forceinline__ float decode_fp8_e4m3fn(uint8_t packed)
{
    if (packed == 0 || packed == 0x80) return 0.0f;
    float sign = (packed & 0x80) ? -1.0f : 1.0f;
    uint32_t exp = (packed >> 3) & 0xF;
    uint32_t mant = packed & 0x7;

    if (exp == 0) {
        float mant_f = mant / 8.0f;
        return sign * mant_f * ldexpf(1.0f, -6);
    } else {
```

```
        float mant_f = 1.0f + mant / 8.0f;
        int e = static_cast<int>(exp) - 7;
        return sign * mant_f * ldexpf(1.0f, e);
    }
}
```

This naive implementation gave ~2000µs — obviously not a good starting point, but I had some optimizations on top of mind: fixing uncoalesced memory access, adding thread collaboration per row, and using warp-level reduction.

Optimization 1: Coalesced Access + Thread Collaboration + Warp Reduction (2000µs → 443µs)

This optimization addressed multiple issues at once:

1. **Coalesced memory access**: Restructured access patterns for better memory throughput
2. **Shared memory for B**: Load B tile into shared memory cooperatively, reused across all rows in block
3. **Multiple threads per row**: Similar to Simon's suggestion in CuTe DSL, multiple threads collaborate on computing each output element
4. **Warp-level reduction**: Use efficient warp shuffle operations for final reduction instead of use shared memory

```
__device__ __forceinline__ half warp_reduce_sum_half(half val) {
    #pragma unroll
    for (int offset = 16; offset > 0; offset >>= 1) {
        val = __hadd(val, __shfl_down_sync(0xffffffff, val, offset));
    }
    return val;
}

constexpr int ROWS_PER_BLOCK = 8;
constexpr int THREADS_PER_ROW = 32;  // One warp per row
constexpr int TILE_K = 128;

// Shared memory for B and SFB tiles (shared across all rows in block)
__shared__ uint8_t smem_B[TILE_K / 2];
```

```
__shared__ uint8_t smem_SFB[TILE_K / 16];

const int tid = threadIdx.x;           // 0..31 (within row)
const int row_in_block = threadIdx.y;  // 0..7
const int linear_tid = row_in_block * THREADS_PER_ROW + tid;
const int total_threads = ROWS_PER_BLOCK * THREADS_PER_ROW;

half local_sum = __float2half(0.0f);

for (int tile = 0; tile < num_k_tiles; ++tile) {
    // Cooperative load of B into shared memory (coalesced)
    for (int i = linear_tid; i < TILE_K / 2; i += total_threads) {
        smem_B[i] = B_base[k_offset / 2 + i];
    }
    if (linear_tid < TILE_K / 16) {
        smem_SFB[linear_tid] = SFB_base[k_offset / 16 + linear_tid];
    }
    __syncthreads();

    // Each thread processes part of K dimension
    for (int k = tid; k < TILE_K / 2; k += THREADS_PER_ROW) {
        // Read A from global memory (coalesced)
        uint8_t a_packed = A_row[k_offset / 2 + k];
        // Read B from shared memory (reused across rows)
        uint8_t b_packed = smem_B[k];
        // ... decode and compute ...
    }
    __syncthreads();
}

local_sum = warp_reduce_sum_half(local_sum);

if (tid == 0) {
    C[batch_idx * M + global_row] = local_sum;
}
```

Optimization 2: Remove Shared Memory + Vectorized Loads + Hardware Intrinsics (443µs → 39µs)

Comparing 443µs with the CuTe DSL's ~33µs, we were clearly missing something big. So I did a NCU profile, which shows the cuda code issuing ~10x more instructions than the CuTe DSL version! This is very likely the big thing we missed so far.

This optimization combined several improvements:

1. **Removed shared memory for B**: Each thread loads B directly — turns out faster without the smem overhead
2. **Each thread handles its own tile**: Instead of collaborating on one tile, each thread processes its own K tiles with vectorized loads
3. **Vectorized loads**: Use `float4` (16 bytes) loads for better memory throughput
4. **Hardware intrinsics**: Use CUDA intrinsics for FP4/FP8 decoding instead of manual bit manipulation

Intrinsics for decoding:

```
__device__ __forceinline__ void decode_fp4_e2m1_half2(uint8_t packed, __half& lo,
{
    __nv_fp4x2_storage_t fp4x2_storage = packed;
    __half2_raw half2_raw = __nv_cvt_fp4x2_to_halfraw2(fp4x2_storage, __NV_E2M1);
    lo = *reinterpret_cast<const __half*>(&half2_raw.x);
    hi = *reinterpret_cast<const __half*>(&half2_raw.y);
}

__device__ __forceinline__ __half decode_fp8_e4m3fn_half(uint8_t packed)
{
    __nv_fp8_storage_t fp8_storage = packed;
    __half_raw half_raw = __nv_cvt_fp8_to_halfraw(fp8_storage, __NV_E4M3);
    return *reinterpret_cast<const __half*>(&half_raw);
}
```

Vectorized loads with each thread handling its own tile:

```
// Each thread processes its own K tiles (no shared memory for B)
for (int tile = tid; tile < num_k_tiles; tile += THREADS_PER_ROW) {
    const int k_offset = tile * TILE_K;

    // Vectorized loads using float4 (16 bytes at a time)
    const float4* A_ptr = reinterpret_cast<const float4*>(A_row + k_offset / 2);
    const float4* B_ptr = reinterpret_cast<const float4*>(B_base + k_offset / 2);

    float4 A_data0 = __ldg(A_ptr);
```

```
    float4 A_data1 = __ldg(A_ptr + 1);
    float4 B_data0 = __ldg(B_ptr);
    float4 B_data1 = __ldg(B_ptr + 1);

    // Load scale factors
    uint32_t sfa_vec = __ldg(reinterpret_cast<const uint32_t*>(SFA_row + k_offset
    uint32_t sfb_vec = __ldg(reinterpret_cast<const uint32_t*>(SFB_base + k_offse

    // Decode and compute using intrinsics...
}
```

The combination of removing shared memory overhead, vectorized loads, and hardware intrinsics brought massive improvement: **443µs → 39µs**!

Optimization 3: PTX Assembly for Vectorized Decode (39µs → ~27µs)

I pushed further with PTX assembly to replace the intrinsics and explicitly do fused multiply-accumulate:

```
__device__ __forceinline__ float decode_mul_accumulate_fp4x8(
    const uint32_t a_packed,
    const uint32_t b_packed,
    float acc)
{
    float result;

    asm volatile (
        "{"
        "  .reg .b8 %%ab<4>, %%bb<4>;\n"
        "  .reg .b32 %%a<4>, %%b<4>;\n"
        "  .reg .b32 %%p0, %%p1;\n"
        "  .reg .f16 %%h0, %%h1;\n"
        "  .reg .f32 %%f0, %%f1;\n"
        "  mov.b32 {%%ab0, %%ab1, %%ab2, %%ab3}, %1;\n"
        "  mov.b32 {%%bb0, %%bb1, %%bb2, %%bb3}, %2;\n"
        "  cvt.rn.f16x2.e2m1x2 %%a0, %%ab0;\n"
        "  cvt.rn.f16x2.e2m1x2 %%a1, %%ab1;\n"
        "  cvt.rn.f16x2.e2m1x2 %%a2, %%ab2;\n"
```

```
        "  cvt.rn.f16x2.e2m1x2 %%a3, %%ab3;\n"
        "  cvt.rn.f16x2.e2m1x2 %%b0, %%bb0;\n"
        "  cvt.rn.f16x2.e2m1x2 %%b1, %%bb1;\n"
        "  cvt.rn.f16x2.e2m1x2 %%b2, %%bb2;\n"
        "  cvt.rn.f16x2.e2m1x2 %%b3, %%bb3;\n"
        "  mul.rn.f16x2 %%p0, %%a0, %%b0;\n"
        "  fma.rn.f16x2 %%p0, %%a1, %%b1, %%p0;\n"
        "  mul.rn.f16x2 %%p1, %%a2, %%b2;\n"
        "  fma.rn.f16x2 %%p1, %%a3, %%b3, %%p1;\n"
        "  add.rn.f16x2 %%p0, %%p0, %%p1;\n"
        "  mov.b32 {%%h0, %%h1}, %%p0;\n"
        "  cvt.f32.f16 %%f0, %%h0;\n"
        "  cvt.f32.f16 %%f1, %%h1;\n"
        "  add.f32 %%f0, %%f0, %%f1;\n"
        "  add.f32 %0, %%f0, %3;\n"
        "}"
        : "=f"(result)
        : "r"(a_packed), "r"(b_packed), "f"(acc)
    );

    return result;
}
```

And for FP8 scale factors:

```
__device__ __forceinline__ void decode_fp8x4_e4m3fn_half4(
    const uint32_t packed, __half& h0, __half& h1, __half& h2, __half& h3)
{
    uint32_t out_low, out_high;

    asm volatile (
        "{"
        "  .reg .b16 %%low, %%high;\n"
        "  mov.b32 {%%low, %%high}, %2;\n"
        "  cvt.rn.f16x2.e4m3x2 %0, %%low;\n"
        "  cvt.rn.f16x2.e4m3x2 %1, %%high;\n"
        "}"
        : "=r"(out_low), "=r"(out_high)
        : "r"(packed)
    );

    __half2 h_low = *reinterpret_cast<const __half2*>(&out_low);
    __half2 h_high = *reinterpret_cast<const __half2*>(&out_high);
```

```
    h0 = h_low.x;
    h1 = h_low.y;
    h2 = h_high.x;
    h3 = h_high.y;
}
```

Optimization 4: Parameter Tuning (~27µs → ~26µs)

Tuned `THREADS_PER_ROW` and `ROWS_PER_BLOCK` (not precise to different problem shapes tune).

Optimization 5: Instruction-Level Parallelism (26µs → ~22.9µs)

Profiling showed long scoreboard stalls — memory loading was the bottleneck. I tried double buffering with async copy, but that didn't help. Then how could we reduce the memory stall? I tried instruction level parallelism and it works for me - **processing 2 tiles per loop iteration**: (As I write to this point, I guess do loop unrolling 2 might give similar effect)

```
// Main loop - process 2 tiles per iteration for better ILP
for (; tile + THREADS_PER_ROW < num_k_tiles; tile += 2 * THREADS_PER_ROW) {
    // Load tile 0
    float4 A_data0_t0 = __ldg(A_ptr_0);
    float4 B_data0_t0 = __ldg(B_ptr_0);
    float4 A_data1_t0 = __ldg(A_ptr_0 + 1);
    float4 B_data1_t0 = __ldg(B_ptr_0 + 1);
    uint32_t sfa_vec_t0 = __ldg(sfa_ptr_0);
    uint32_t sfb_vec_t0 = __ldg(sfb_ptr_0);

    // Load tile 1 (overlapped with tile 0 loads)
    float4 A_data0_t1 = __ldg(A_ptr_1);
    float4 B_data0_t1 = __ldg(B_ptr_1);
    float4 A_data1_t1 = __ldg(A_ptr_1 + 1);
    float4 B_data1_t1 = __ldg(B_ptr_1 + 1);
    uint32_t sfa_vec_t1 = __ldg(sfa_ptr_1);
    uint32_t sfb_vec_t1 = __ldg(sfb_ptr_1);

    // Process tile 0 (data ready due to load latency)
    process_tile(...);
```

```
    // Process tile 1
    process_tile(...);
}
```

I tried 3 or 4 tiles per loop, but those were slightly slower. My hypothesis was use more tiles use more register so cause register spills, however, profiling with `lineinfo` showed register number doesn't increase significantly and still kept the same occupancy, but I saw some loads stalling on previous loads — likely due to register reuse by the compiler I guess.

Optimization 6: Aggressive PTX Fusion (22.9µs → ~22.3µs)

The final optimization fused everything into one large PTX block — decode A, B, scales, and all multiplications and adds:

```
__device__ __forceinline__ void process_tile(
    float& local_sum,
    const uint32_t A0_0, const uint32_t A0_1, const uint32_t A0_2, const uint32_t
    const uint32_t A1_0, const uint32_t A1_1, const uint32_t A1_2, const uint32_t
    const uint32_t B0_0, const uint32_t B0_1, const uint32_t B0_2, const uint32_t
    const uint32_t B1_0, const uint32_t B1_1, const uint32_t B1_2, const uint32_t
    const uint32_t sfa_packed,
    const uint32_t sfb_packed)
{
    asm volatile (
       "{"
       "  .reg .b16 %%sfalo, %%sfahi, %%sfblo, %%sfbhi;\\n"
       "  .reg .b32 %%sa01, %%sa23, %%sb01, %%sb23;\\n"
       "  .reg .b32 %%scale01, %%scale23;\\n"
       "  .reg .f32 %%s0, %%s1, %%s2, %%s3;\\n"
       "  .reg .b8 %%a<4>, %%b<4>;\\n"
       "  .reg .b32 %%fa<4>, %%fb<4>;\\n"
       "  .reg .b32 %%p0, %%p1, %%p2, %%p3;\\n"
       "  .reg .f16 %%h0, %%h1;\\n"
       "  .reg .f32 %%f0, %%f1, %%acc0, %%acc1, %%acc2, %%acc3, %%tile_result, %%

       "  mov.f32 %%one, 0f3f800000;\\n"
       "  mov.b32 {%%sfalo, %%sfahi}, %17;\\n"
       "  mov.b32 {%%sfblo, %%sfbhi}, %18;\\n"
       "  cvt.rn.f16x2.e4m3x2 %%sa01, %%sfalo;\\n"
```

```
        "  cvt.rn.f16x2.e4m3x2 %%sa23, %%sfahi;\\n"
        "  cvt.rn.f16x2.e4m3x2 %%sb01, %%sfblo;\\n"
        "  cvt.rn.f16x2.e4m3x2 %%sb23, %%sfbhi;\\n"
        "  mul.rn.f16x2 %%scale01, %%sa01, %%sb01;\\n"
        "  mul.rn.f16x2 %%scale23, %%sa23, %%sb23;\\n"

        "  mov.b32 {%%h0, %%h1}, %%scale01;\\n"
        "  cvt.f32.f16 %%s0, %%h0;\\n"
        "  cvt.f32.f16 %%s1, %%h1;\\n"
        "  mov.b32 {%%h0, %%h1}, %%scale23;\\n"
        "  cvt.f32.f16 %%s2, %%h0;\\n"
        "  cvt.f32.f16 %%s3, %%h1;\\n"

        "  mov.b32 {%%a0, %%a1, %%a2, %%a3}, %1;\\n"
        "  mov.b32 {%%b0, %%b1, %%b2, %%b3}, %9;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa0, %%a0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa1, %%a1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa2, %%a2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa3, %%a3;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb0, %%b0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb1, %%b1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb2, %%b2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb3, %%b3;\\n"
        "  mul.rn.f16x2 %%p0, %%fa0, %%fb0;\\n"
        "  fma.rn.f16x2 %%p0, %%fa1, %%fb1, %%p0;\\n"
        "  fma.rn.f16x2 %%p0, %%fa2, %%fb2, %%p0;\\n"
        "  fma.rn.f16x2 %%p0, %%fa3, %%fb3, %%p0;\\n"


        "  mov.b32 {%%a0, %%a1, %%a2, %%a3}, %2;\\n"
        "  mov.b32 {%%b0, %%b1, %%b2, %%b3}, %10;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa0, %%a0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa1, %%a1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa2, %%a2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa3, %%a3;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb0, %%b0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb1, %%b1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb2, %%b2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb3, %%b3;\\n"
        "  fma.rn.f16x2 %%p0, %%fa0, %%fb0, %%p0;\\n"
        "  fma.rn.f16x2 %%p0, %%fa1, %%fb1, %%p0;\\n"
        "  fma.rn.f16x2 %%p0, %%fa2, %%fb2, %%p0;\\n"
        "  fma.rn.f16x2 %%p0, %%fa3, %%fb3, %%p0;\\n"
```

```
        "  mov.b32 {%%h0, %%h1}, %%p0;\\n"
        "  cvt.f32.f16 %%f0, %%h0;\\n"
        "  cvt.f32.f16 %%f1, %%h1;\\n"
        "  add.f32 %%acc0, %%f0, %%f1;\\n"
        "  mul.f32 %%acc0, %%acc0, %%s0;\\n"

        "  mov.b32 {%%a0, %%a1, %%a2, %%a3}, %3;\\n"
        "  mov.b32 {%%b0, %%b1, %%b2, %%b3}, %11;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa0, %%a0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa1, %%a1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa2, %%a2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa3, %%a3;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb0, %%b0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb1, %%b1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb2, %%b2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb3, %%b3;\\n"
        "  mul.rn.f16x2 %%p1, %%fa0, %%fb0;\\n"
        "  fma.rn.f16x2 %%p1, %%fa1, %%fb1, %%p1;\\n"
        "  fma.rn.f16x2 %%p1, %%fa2, %%fb2, %%p1;\\n"
        "  fma.rn.f16x2 %%p1, %%fa3, %%fb3, %%p1;\\n"
        "  mov.b32 {%%a0, %%a1, %%a2, %%a3}, %4;\\n"
        "  mov.b32 {%%b0, %%b1, %%b2, %%b3}, %12;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa0, %%a0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa1, %%a1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa2, %%a2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa3, %%a3;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb0, %%b0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb1, %%b1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb2, %%b2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb3, %%b3;\\n"
        "  fma.rn.f16x2 %%p1, %%fa0, %%fb0, %%p1;\\n"
        "  fma.rn.f16x2 %%p1, %%fa1, %%fb1, %%p1;\\n"
        "  fma.rn.f16x2 %%p1, %%fa2, %%fb2, %%p1;\\n"
        "  fma.rn.f16x2 %%p1, %%fa3, %%fb3, %%p1;\\n"
        "  mov.b32 {%%h0, %%h1}, %%p1;\\n"
        "  cvt.f32.f16 %%f0, %%h0;\\n"
        "  cvt.f32.f16 %%f1, %%h1;\\n"
        "  add.f32 %%acc1, %%f0, %%f1;\\n"
        "  fma.rn.f32 %%acc0, %%acc1, %%s1, %%acc0;\\n"

        "  mov.b32 {%%a0, %%a1, %%a2, %%a3}, %5;\\n"
        "  mov.b32 {%%b0, %%b1, %%b2, %%b3}, %13;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa0, %%a0;\\n"
```

```
        "  cvt.rn.f16x2.e2m1x2 %%fa1, %%a1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa2, %%a2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa3, %%a3;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb0, %%b0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb1, %%b1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb2, %%b2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb3, %%b3;\\n"
        "  mul.rn.f16x2 %%p2, %%fa0, %%fb0;\\n"
        "  fma.rn.f16x2 %%p2, %%fa1, %%fb1, %%p2;\\n"
        "  fma.rn.f16x2 %%p2, %%fa2, %%fb2, %%p2;\\n"
        "  fma.rn.f16x2 %%p2, %%fa3, %%fb3, %%p2;\\n"
        "  mov.b32 {%%a0, %%a1, %%a2, %%a3}, %6;\\n"
        "  mov.b32 {%%b0, %%b1, %%b2, %%b3}, %14;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa0, %%a0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa1, %%a1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa2, %%a2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa3, %%a3;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb0, %%b0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb1, %%b1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb2, %%b2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb3, %%b3;\\n"
        "  fma.rn.f16x2 %%p2, %%fa0, %%fb0, %%p2;\\n"
        "  fma.rn.f16x2 %%p2, %%fa1, %%fb1, %%p2;\\n"
        "  fma.rn.f16x2 %%p2, %%fa2, %%fb2, %%p2;\\n"
        "  fma.rn.f16x2 %%p2, %%fa3, %%fb3, %%p2;\\n"
        "  mov.b32 {%%h0, %%h1}, %%p2;\\n"
        "  cvt.f32.f16 %%f0, %%h0;\\n"
        "  cvt.f32.f16 %%f1, %%h1;\\n"
        "  add.f32 %%acc2, %%f0, %%f1;\\n"
        "  fma.rn.f32 %%acc0, %%acc2, %%s2, %%acc0;\\n"

        "  mov.b32 {%%a0, %%a1, %%a2, %%a3}, %7;\\n"
        "  mov.b32 {%%b0, %%b1, %%b2, %%b3}, %15;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa0, %%a0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa1, %%a1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa2, %%a2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa3, %%a3;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb0, %%b0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb1, %%b1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb2, %%b2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb3, %%b3;\\n"
        "  mul.rn.f16x2 %%p3, %%fa0, %%fb0;\\n"
        "  fma.rn.f16x2 %%p3, %%fa1, %%fb1, %%p3;\\n"
```

```
        "  fma.rn.f16x2 %%p3, %%fa2, %%fb2, %%p3;\\n"
        "  fma.rn.f16x2 %%p3, %%fa3, %%fb3, %%p3;\\n"
        "  mov.b32 {%%a0, %%a1, %%a2, %%a3}, %8;\\n"
        "  mov.b32 {%%b0, %%b1, %%b2, %%b3}, %16;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa0, %%a0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa1, %%a1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa2, %%a2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fa3, %%a3;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb0, %%b0;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb1, %%b1;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb2, %%b2;\\n"
        "  cvt.rn.f16x2.e2m1x2 %%fb3, %%b3;\\n"
        "  fma.rn.f16x2 %%p3, %%fa0, %%fb0, %%p3;\\n"
        "  fma.rn.f16x2 %%p3, %%fa1, %%fb1, %%p3;\\n"
        "  fma.rn.f16x2 %%p3, %%fa2, %%fb2, %%p3;\\n"
        "  fma.rn.f16x2 %%p3, %%fa3, %%fb3, %%p3;\\n"
        "  mov.b32 {%%h0, %%h1}, %%p3;\\n"
        "  cvt.f32.f16 %%f0, %%h0;\\n"
        "  cvt.f32.f16 %%f1, %%h1;\\n"
        "  add.f32 %%acc3, %%f0, %%f1;\\n"
        "  fma.rn.f32 %%tile_result, %%acc3, %%s3, %%acc0;\\n"
        "  fma.rn.f32 %0, %%tile_result, %%one, %0;\\n"
        "}"
        : "+f"(local_sum)
        : "r"(A0_0), "r"(A0_1), "r"(A0_2), "r"(A0_3),
          "r"(A1_0), "r"(A1_1), "r"(A1_2), "r"(A1_3),
          "r"(B0_0), "r"(B0_1), "r"(B0_2), "r"(B0_3),
          "r"(B1_0), "r"(B1_1), "r"(B1_2), "r"(B1_3),
          "r"(sfa_packed), "r"(sfb_packed)
    );

}
```

This kernel achieved **~22.3µs**, which was my final submission.

Performance Summary

Implementation	Time
CuTe DSL Template	100µs

Implementation	Time
CuTe DSL Optimized	~33µs
CUDA Naive	2000µs
CUDA + Coalesced Access + Thread Collaboration	443µs
CUDA + Hardware Intrinsics	39µs
CUDA + PTX	~27µs
CUDA + ILP (2 tiles)	~22.9µs
CUDA + Aggressive PTX Fusion	~22.3µs

Future Optimization Ideas

Things I didn't get a chance to try:

1. **Tune TILE_K**: Currently hardcoded at 64. Different tile sizes might perform better for different problem shapes, but would require PTX code changes.
2. **Try double buffering again with different TILE_K**: Async copy with pipelining might help with a different tile configuration.
3. **Template specialization per problem shape**: Create optimized kernels for each (M, K, L) combination in the benchmark.

It's been a great learning journey getting hands-on with NVFP4 on Blackwell GPUs and learning CuTe DSL. Thanks to GPU Mode for organizing this hackathon!

Full Code

Full code for all implementations will be available in my GitHub repository (link coming soon).

References

- Blackwell NVFP4 Kernel Hackathon Event
- GPU Mode Leaderboard

- Reference Kernels Repository
- Simon Veitner's NVFP4 GEMV Blog
- NVIDIA CUDA Math API - FP4 Intrinsics
- NVIDIA PTX ISA Documentation

Yue's Learning Journey

Yue's Learning Journey

Notes and thoughts on LLM, GPU Kernels, and everything I'm learning